#LEARNINGS: A video is not a single image but a sequence of images, and each frame is represented as a NumPy array with dimensions indicating height, width, and color channels. It is important to always verify whether the video is successfully opened and whether each frame is properly read before accessing its properties like shape and consume significant memory. Limiting frame processing, checking return values, and resizing frames when necessary are essential techniques for stable execution. RGB for matplotlib BGR for opencv,CYMK (colourcoding) cap=cv2.videocapture(function) plt.imshow(img) subject image to plt

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm

#import subprocess

ipd.Video("026c7465-309f6d33.mp4",width = 200)
```

```
<IPython.core.display.Video object>
```

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in
/usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in
/usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```
# Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')

# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
2398.0
```

```
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
Height 720.0, Width 1280.0
```

```
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

```
FPS : 59.94
```

```
cap.release()
```

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read()                                #will return TRUE
everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

```
Returned True and img of shape (720, 1280, 3)
```

```
plt.imshow(img)  #opencv loads color in BGR format, matplot lib
expects RGB format
```

```
<matplotlib.image.AxesImage at 0x7ab7a738d4f0>
```

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read()                                #will return TRUE
everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

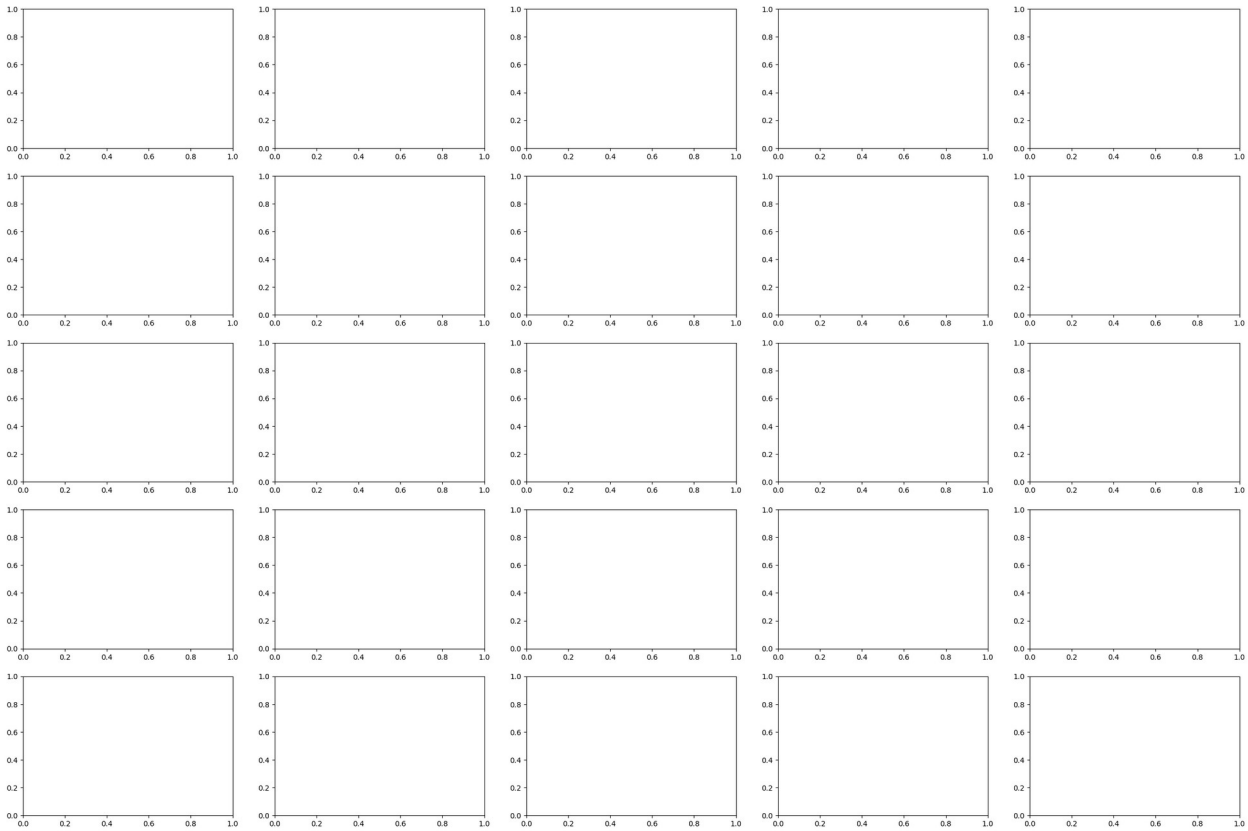
```
Returned True and img of shape (720, 1280, 3)
```

```
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")

display_cv2_img(img)
```

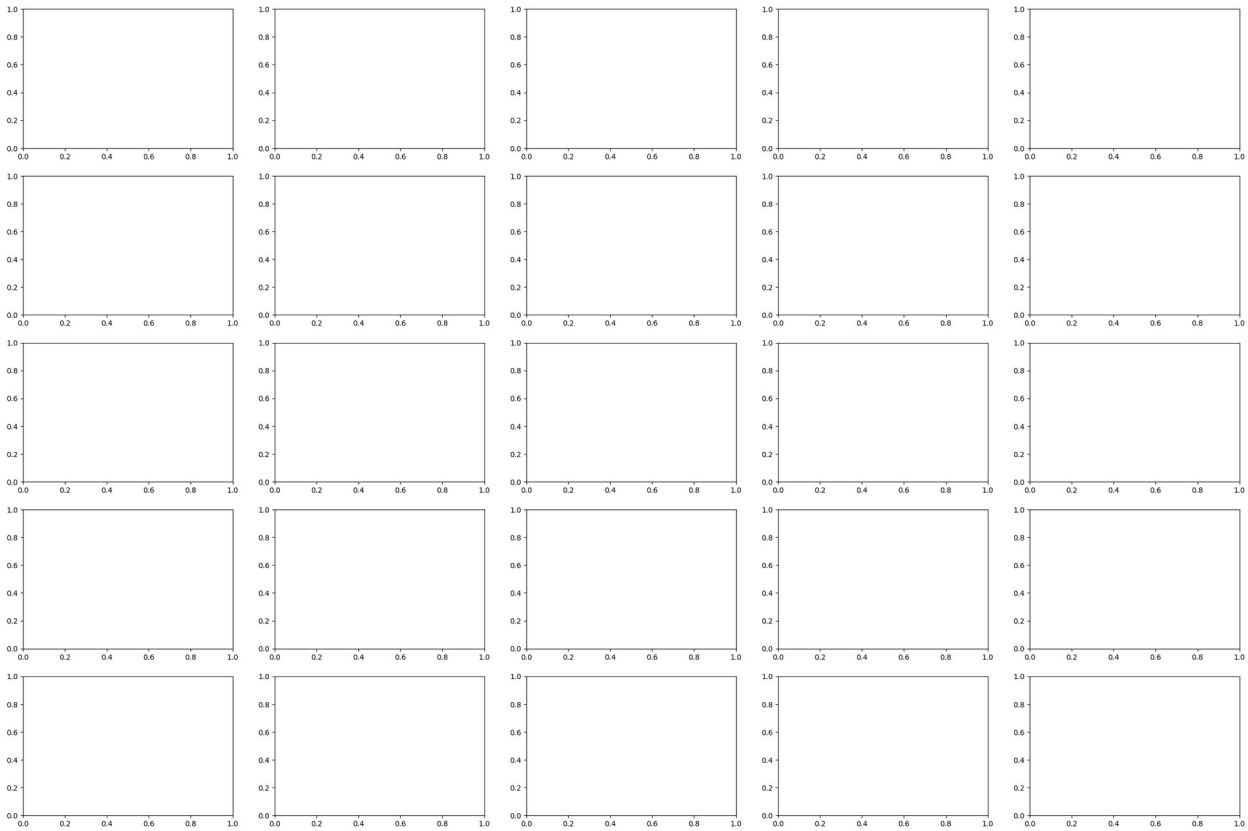
```
cap.release()

fig, axs = plt.subplots(5, 5, figsize=(30, 20))
plt.show()
```



```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()
plt.show()

# Without flatten → 2D grid (row, column)
# With flatten → 1D list (single index)
```



```
import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:
#frame is divisible by 100 - Note the modulo operator
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()
#automatically adjusts the spacing between the plots
plt.show()
cap.release()
```

```
labels = pd.read_csv('mot_labels.csv',low_memory=False)
##labels.head()
#video_labels = (labels.query('videoName == "026c7465-
309f6d33"').reset_index(drop=True).copy())
#video_labels["video_frame"] = (video_labels["frameIndex"] *
11.9).round().astype("int") #Video is at 60Hz but the labellers did
only 5 Hz
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-1597131570.py in <cell line: 0>()
----> 1 labels = pd.read_csv('mot_labels.csv',low_memory=False)
      2 ##labels.head()
      3 #video_labels = (labels.query('videoName == "026c7465-
309f6d33"').reset_index(drop=True).copy())
      4 #video_labels["video_frame"] = (video_labels["frameIndex"] *
11.9).round().astype("int") #Video is at 60Hz but the labellers did
only 5 Hz

/usr/local/lib/python3.12/dist-packages/pandas/io/parsers/readers.py
in read_csv(filepath_or_buffer, sep, delimiter, header, names,
index_col, usecols, dtype, engine, converters, true_values,
false_values, skipinitialspace, skiprows, skipfooter, nrows,
```

```
na_values, keep_default_na, na_filter, verbose, skip_blank_lines,
parse_dates, infer_datetime_format, keep_date_col, date_parser,
date_format, dayfirst, cache_dates, iterator, chunksize, compression,
thousands, decimal, lineterminator, quotechar, quoting, doublequote,
escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines,
delim_whitespace, low_memory, memory_map, float_precision,
storage_options, dtype_backend)
   1024     kwds.update(kwds_defaults)
   1025
-> 1026     return _read(filepath_or_buffer, kwds)
   1027
   1028

/usr/local/lib/python3.12/dist-packages/pandas/io/parsers/readers.py
in _read(filepath_or_buffer, kwds)
    618
    619     # Create the parser.
--> 620     parser = TextFileReader(filepath_or_buffer, **kwds)
    621
    622     if chunksize or iterator:

/usr/local/lib/python3.12/dist-packages/pandas/io/parsers/readers.py
in __init__(self, f, engine, **kwds)
   1618
   1619         self.handles: IOHandles | None = None
-> 1620         self._engine = self._make_engine(f, self.engine)
   1621
   1622     def close(self) -> None:

/usr/local/lib/python3.12/dist-packages/pandas/io/parsers/readers.py
in _make_engine(self, f, engine)
   1878                 if "b" not in mode:
   1879                     mode += "b"
-> 1880             self.handles = get_handle(
   1881                 f,
   1882                 mode,

/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory:
'mot_labels.csv'

#print(video_labels.shape)
```

```python
print(labels["category"].value_counts())

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame == 1035:
        break
cap.release()

display_cv2_img(img)
```